

RELATÓRIO TÉCNICO

Dark Roads: Byteland

T290-16

Aaron Magno, Arthur Soares e Davi de Paula

Engenharia da Modelagem

A conversão do problema físico em abstração matemática.

Da Narrativa ao Grafo



Abstraímos a malha viária urbana para um **Grafo Ponderado e Não Direcionado**:

- ✓ **Vértices:** Representam as junções da cidade.
- ✓ **Arestas:** Representam as estradas bidirecionais.
- ✓ **Pesos:** Metragem da via (equivalente ao custo operacional).



Objetivo: Árvore Geradora Mínima



Segurança

A restrição fundamental é manter a conectividade total: todas as junções devem ser alcançáveis entre si.



Maximização

O lucro é obtido desligando luzes redundantes. Buscamos a estrutura de custo mínimo (MST).

$$\text{Economia} = \Sigma(\text{Custo Original}) - \Sigma(\text{Custo MST})$$

Estratégia: Algoritmo de Kruskal



Ordenação Global

Análise de arestas do menor para o maior peso (abordagem Greedy).



Grafo Esparso

Justificativa: Melhor desempenho para redes de estradas onde $E \ll V^2$.



Simplicidade

Processamento direto da lista bruta de arestas sem overhead de adjacência.

Union-Find / DSU

Gerenciamento de componentes para detecção de ciclos:

- ✓ **Find:** Identifica se junções já estão conectadas.
- ✓ **Union:** Une componentes ao aceitar uma aresta.
- ✓ **Compressão:** Otimiza buscas para tempo quase constante.

```
class UF {  
    private int[] id;  
  
    public UF(int n) {  
        id = new int[n];  
        for (int i = 0; i < n; i++) {  
            id[i] = i;  
        }  
    }  
  
    public int find(int p) {  
        int raiz = p;  
        while (raiz != id[raiz]) {  
            raiz = id[raiz];  
        }  
        // Compressão de caminho (Path Compression)  
        int atual = p;  
        while (atual != raiz) {  
            int proximo = id[atual];  
            id[atual] = raiz;  
            atual = proximo;  
        }  
        return raiz;  
    }  
  
    public boolean union(int p, int q) {  
        int raizP = find(p);  
        int raizQ = find(q);  
        if (raizP == raizQ) {  
            return false;  
        }  
        id[raizP] = raizQ;  
        return true;  
    }  
}
```

Análise de Performance

Complexidade de Tempo e Espaço

- Tempo: $O(E \log E)$ — Gargalo focado na ordenação das 200.000 arestas.
- Operações: $O(E \alpha(V))$ — DSU com complexidade quase constante.
- Memória: $O(V + E)$ — Alocação linear baseada no array bruto de arestas.
- **Nota: Uso obrigatório de `long` para evitar Overflow no cálculo total.**

Casos-Limite e Robustez

- ✓ **Pesos Iguais:** Geram múltiplas MSTs válidas; custo final invariante.
- ✓ **Conectividade:** Enunciado garante grafo conexo; Kruskal cobre todos os vértices.
- ✓ **Múltiplos Testes:** Reset completo das estruturas IDs e componentes a cada iteração.